GBIN5U03 - Principes de programmation et outils de développement

Guillaume Huard

Université Grenoble Alpes

Plan

Présentation de l'UE

Le langage C

Elements consitutifs du langage C

Programmation modulaire

Compilation séparée

Environnement de programmation

Informations factuelles

Intervenants (`Prénom.Nom@univ-grenoble-alpes.fr`)

- Guillaume Huard (responsable)
- Valentin Garnero
- Farid Lassoued
- Valentin Trophime

Organisation

- 1 créneau hébdomadaire de cours OU TP (cf. ADE)
- 2 apnées dans le semestre (pas de rendu, non évalué, questions à adresser directement à l'enseignant de TP concerné)
- projet logiciel de fin de semestre

Evaluation ($max(\frac{1}{20}CC + \frac{10}{20}ET, \frac{11}{20}ET) + \frac{9}{20}Projet$)

- 2 activités de programmation individuelles sur caséine, CC
- Examen terminal, ET, deux sessions
- Projet, poids "modéré" car il s'agit d'une note de groupe

Forme

Semestre : révisions, approfondissement, préparation au projet

- langage C et principes de programmation
- programmation shell
- outils de développement

Projet : simulateur ARM

- deux semaines à temps plein
- travail en groupe
- exploitation de documentation technique

Mode d'évaluation pour l'ET

Examen de programmation sur machine

- nouveauté pour cette année
- organisation lourde
  - 2nd étage de l'UFR réservé en entier (11 salles de TP)
  - autour de 11 surveillants
- préparation indispensable
  - comptes anonymes, configuration de l'UFR
  - déroulement similaire aux activités de CC
  - soyez efficaces dans votre cycle de développement

Plan

Présentation de l'UE

Le langage C

Elements consitutifs du langage C

Programmation modulaire

Compilation séparée

Environnement de programmation

Origine

Créé en 1972 par K. Thompson, D. Ritchie, B. Kernighan

- Aux débuts d'UNIX (1969) : développement du système
- Encore largement utilisé aujourd'hui
  - Noyau des systèmes d'exploitation
  - Programmes nécessitant une grande efficacité de calcul
  - Systèmes embarqués (peu de mémoire, runtime basique)
- Etendu en C++ par B. Stroustrup en 1983, notamment
  - Orienté objet
  - Exception
  - Polymorphisme paramétré
  - Références

Mon premier programme en C

Tout programme doit contenir une fonction main

```c
// Un commentaire débute par //
// #include va chercher le contenu d'un fichier
// stdio.h : affichage et lecture standards
#include <stdio.h>
int main() {
    // printf permet d'afficher à l'écran
    // \n est le caractère de fin de ligne
    printf("Hello world\n");
    // Toutes les instructions se terminent par un ;
    return 0;
}
```

Rappel : Dans un environnement UNIX, un programme renvoie :

- 0, si tout s'est bien passé
- Un code d'erreur non nul, en cas de problème

Compilation

Traduction du code source en un programme exécutable

```
clang -o programme -Wall -Werror -g hello_world.c
```

Options utiles

- `-o nom` : nom du fichier exécutable
- `-Wall` : afficher plus d'avertissements
- `-Werror` : considérer les avertissements comme des erreurs
- `-g` : inclure les infos de deboguage

Quelques compilateurs : `gcc`, `llvm-gcc`, `clang`, `icc`

Plan

Présentation de l'UE

Le langage C

Elements consitutifs du langage C

Programmation modulaire

Compilation séparée

Environnement de programmation

Le typage

Toutes les variables ont un type

- déclarer une variable avant de l'utiliser
- le type précise la nature et la taille du contenu de la variable
- le compilateur vérifie le typage
  - paramètres effectifs des appels de fonctions
  - opérandes des opérateurs utilisés
- lorsque les types de correspondent pas
  - conversion automatique si possible
  - erreur sinon

Mise en œuvre

Plusieurs types de base

- entiers : `char`, `short`, `int`, `long`, `long long` et les variantes `unsigned`
- rationnels : `float`, `double`
- ...

```c
int a = 0;

int main() {
    for (int i = 0; i < 10; i++) {
        a++;
        printf("%d ", a);
    }
    printf("\n");
    return 0;
}
```

Valeurs et typage

```
// déclaration d'une variable x dans Z
int x;
// déclaration d'une variable y dans Q
float y;

x = 1000000000;    // Trop gros ou pas ?
y = 1.0/3.0;       // Quelle précision ?
```

Beaucoup de conversions automatiques autorisées

- grands entiers dans petits entiers
- signé dans non signé et inversement
- flottant dans entier et inversement

Le compilateur peut signaler les conversions implicites dangereuses

```
clang -o programme -Wconversion programme.c
```

Les types entiers

Caractéristiques

- taille (nombre de bits) non standardisée en C :
  peut varier selon l'architecture/OS
- signés par défaut (entiers relatifs),
  `unsigned` pour une variante non signée (entier naturel)

Types entiers portables dans `stdint.h`

taille	$\geq$ dans standard	dans `stdint.h`	`relatif`
8	`char`	`int8_t`	`uint8_t`
16	`short, int`	`int16_t`	`uint16_t`
32	`long`	`int32_t`	`uint32_t`
64	`long long`	`int64_t`	`uint64_t`
adresse	?	`intptr_t`	

Les booléens

Pas de type booléen en C avant 1999
Type de l'expression `a <= b` ?
$\Rightarrow$ `int`

- "faux" est représenté par la valeur `0`
- "vrai" est représenté par toute valeur différente de `0`

Les opérateurs booléens sont définis sur les entiers et utilisent généralement 0 et 1 pour faux et vrai :

- et booléen `&&`
- ou booléen `||`
- négation `!`

Les caractères

Pas de type spécifique aux caractères en C

- on manipule des codes entiers qui les représentent
- traduction à la lecture, à l'écriture ou dans les constantes

Standard ASCII associe un caractère à chaque entier de [0 ;127]

- 128 caractères définis $\Rightarrow$ 7 bits de stockage
- un octet suffit
- 8ème bit utilisé dans diverses extensions, ex :
  - Windows latin-1
  - Mac OS roman

Entrée / sorties

Deux fonctions

- `printf` : affiche la valeur des paramètres
- `scanf` : lit la valeur à donner aux paramètres
  - implique un passage par adresse, explicite en C
  - les arguments sont précédés de l'opérateur &

Le format (1er paramètre) indique comment interpréter les autres paramètres

```
int main() {
        int i;

        scanf(" %d", &i);
        printf("%d %x %c\n", i, i, i);
        return 0;
}
```

Format

Chaine de caractères, contenant des

- parties constantes à afficher ou atendues lors de la lecture
- directives de traitement, une par argument suivant

Directives classiques

- entiers : `%d` et `%i` (décimal), `%x` (hexadécimal), `%c` (caractère)
- variantes : `%ld` (entier long), `%u` et `%lu` (non signés)
- rationnels : `%f` (float), `%lf` (double)
- chaines de caractères : `%s`

Cas particulier pour `scanf`

- `" "` est une séquence quelconque de séparateurs
- `%s` lit un seul mot (jusqu'au prochain séparateur)

Tableaux

Ensemble d'éléments consécutifs de même type, dont le nombre

- est la taille du tableau, donnée à la déclaration
- ne varie pas au cours de l'exécution
- n'est pas stockée par le C (à nous de le faire)

Exemple

```
int t[10];                    // Déclaration

for (int i=0; i<10; i++) { // Indices dans [0;taille-1]
        t[i] = 1;                    // Ecriture
        for (int j=i-1; j>0; j--)
                t[j] = t[j] + t[j-1];
        for (int j=0; j<=i; j++)
                printf("%d ", t[j]);    // Lecture
        printf("\n");
}
```

Débordements

```
int tableau[10];
tableau[99] = 42;
```

Le C ne vérifie pas que l'indice donné est dans le tableau

- la commande ci-dessus marche
  - le programme va écrire dans la mémoire à la case demandée
  - c'est au delà de la fin du tableau
- deux cas de figure pour l'erreur
  - la case est en dehors de la mémoire octroyée à l'utilisateur
    $\Rightarrow$ `Segmentation fault`
  - sinon, contient potentiellement autre chose, son contenu sera écrasé, bug très insidieux
- toujours vérifier qu'on accède bien aux cases de 0 à $taille - 1$

Fonction

Caractérisée par son nom et :

- le type de sa valeur de retour (`void` si aucune)
- le type et le nom de chacun de ses paramètres formels
- sa définition qui peut être séparée de sa déclaration

Prototype : déclarer une fonction avant sa définition

```
int addition(int, int); // prototype

int main() {
        int a, b;
        scanf ("%d %d", &a, &b);
        printf("%d %d %d\n", a, b, addition(a,b));
        return 0;
}

int addition(int a, int b) { // définition
        a += b;
        return a;
}
```

Passage de paramètres

Par valeur

- Emplacement mémoire créé pour les paramètres formels
- La valeur des paramètres effectifs y est copiée

=> les variables passées à une fonction restent inchangées

```
#include <stdio.h>
void echange(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
int main() {
    int qd = 42;       int ue = 503;
    echange(qd, ue);
    printf("En %d on parle de %d\n", ue, qd);
    return 0;
}
```

Le cas des tableaux

Exception à la règle, passés par adresse

```c
#include <stdio.h>
void echange(int Tab[], int i, int j) {
    int tmp;
    tmp = Tab[i];
    Tab[i] = Tab[j];
    Tab[j] = tmp;
}
int main() {
    // Initialisation de taille et valeurs
    int Tab[] = {42, 503};
    echange(Tab, 0, 1);
    printf("En %d on parle de %d\n", Tab[0], Tab[1]);
    return 0;
}
```

Chaîne de caractères

Tableau de caractères contenant le caractère '\0' (convention)

- tableaux passés par adresse, pas de & pour `scanf("%s", s);`
- attention aux débordements !

```
int my_strlen(char s[]) {
        int i=0;

        while (s[i] != '\0')
                i++;
        return i;
}
int main() {
        char s[10];

        scanf("%s", s);
        printf("%d\n", my_strlen(s));
        return 0;
}
```

Dans la bibliothèque standard (`strlen`, `strcpy`, `strcmp`, ...)

Structure

Type composé de plusieurs parties (champs) de types hétérogènes

- opérateur . pour l'accès aux champs d'une variable
- affectation et passage en paramètre par copie

```
struct complexe {
        double r, i;
};
struct complexe addition(struct complexe a,
                         struct complexe b) {
        a.r += b.r; a.i += b.i;
        return a;
}
int main() {
        struct complexe a, b, c;
        a.r = 2.0; a.i = 1.3;
        b.r = 1.1; b.i = 0.5;
        c = addition(a, b);
        printf("%lf %lf %lf", a.i, b.i, c.i);
        return 0;
}
```

Type

Défini par typedef à partir de toute construction

```
typedef struct {
        double r, i;
} complexe;

complexe addition(complexe a, complexe b) {
        a.r += b.r; a.i += b.i;
        return a;
}

int main() {
        complexe a, b, c;
        a.r = 2.0; a.i = 1.3;
        b.r = 1.1; b.i = 0.5;
        c = addition(a, b);
        printf("%lf %lf %lf\n", a.i, b.i, c.i);
        return 0;
}
```

Plan

Présentation de l'UE

Le langage C

Elements consitutifs du langage C

Programmation modulaire

Compilation séparée

Environnement de programmation

Abstraction plus poussée

```c
typedef struct { double r, i; } complexe;

complexe creer_complexe(double r, double i) {
        complexe c; c.r = r; c.i = i; return c;
}

double img(complexe c) { return c.i; }

complexe addition(complexe a, complexe b) {
        a.r += b.r; a.i += b.i; return a;
}

int main() {
        complexe a, b, c;
        a = creer_complexe(2.0, 1.3);
        b = creer_complexe(1.1, 0.5);
        c = addition(a, b);
        printf("%lf %lf %lf\n", img(a), img(b), img(c));
        return 0;
}
```

Programmation modulaire

Consiste à séparer une partie indépendante du programme

- interface d'utilisation placée dans un fichier `.h`
- implémentation confinée dans un fichier `.c`

Multiples avantages

- distribution du travail aisée, selon le découpage en modules
- l'interface simplifie le dialogue entre les développeurs
- compilation séparée possible
- evolutivité des modules, changement d'implémentation possible
- test simplifié, par module, de manière incrémentale

Interface : fichier d'entête (.h pour *header*)

Déclarations (types, variables externes, prototypes)

complexe.h

```c
#ifndef __COMPLEXE_H__ // directive du préprocesseur
#define __COMPLEXE_H__ // évite l'inclusion multiple
typedef struct {
  double r, i;
} complexe;

// constructeur
complexe creer_complexe(double, double);

// accesseur
double img(complexe);

//operation
complexe addition(complexe, complexe);
#endif
```

Implementation : fichier source (.c comme le langage)

Définitions des fonctions et variables annoncées dans l'interface

complexe.c

```c
#include "complexe.h"

complexe creer_complexe(double r, double i) {
  complexe c;
  c.r = r; c.i = i;
  return c;
}
double img(complexe c) {
  return c.i;
}
complexe addition(complexe a, complexe b) {
  a.r += b.r; a.i += b.i;
  return a;
}
```

Programme principal

Le programme principal n'a besoin que de l'interface

main.c

```c
#include <stdio.h>
#include "complexe.h"

int main() {
  complexe a, b, c;
  a = creer_complexe(2.0, 1.3);
  b = creer_complexe(1.1, 0.5);
  c = addition(a, b);
  printf("%lf %lf %lf\n", img(a), img(b), img(c));
  return 0;
}
```

Plan

Présentation de l'UE

Le langage C

Elements consitutifs du langage C

Programmation modulaire

Compilation séparée

Environnement de programmation

Compilation séparée

Deux étapes pour compiler

- traduction et optimisation du code de chaque fichier source
- regroupement de l'ensemble en un unique programme

Avec clang

- traduction d'un fichier source `.c` en fichier objet `.o`

```
clang -Wall -Werror -g -c complexe.c
```

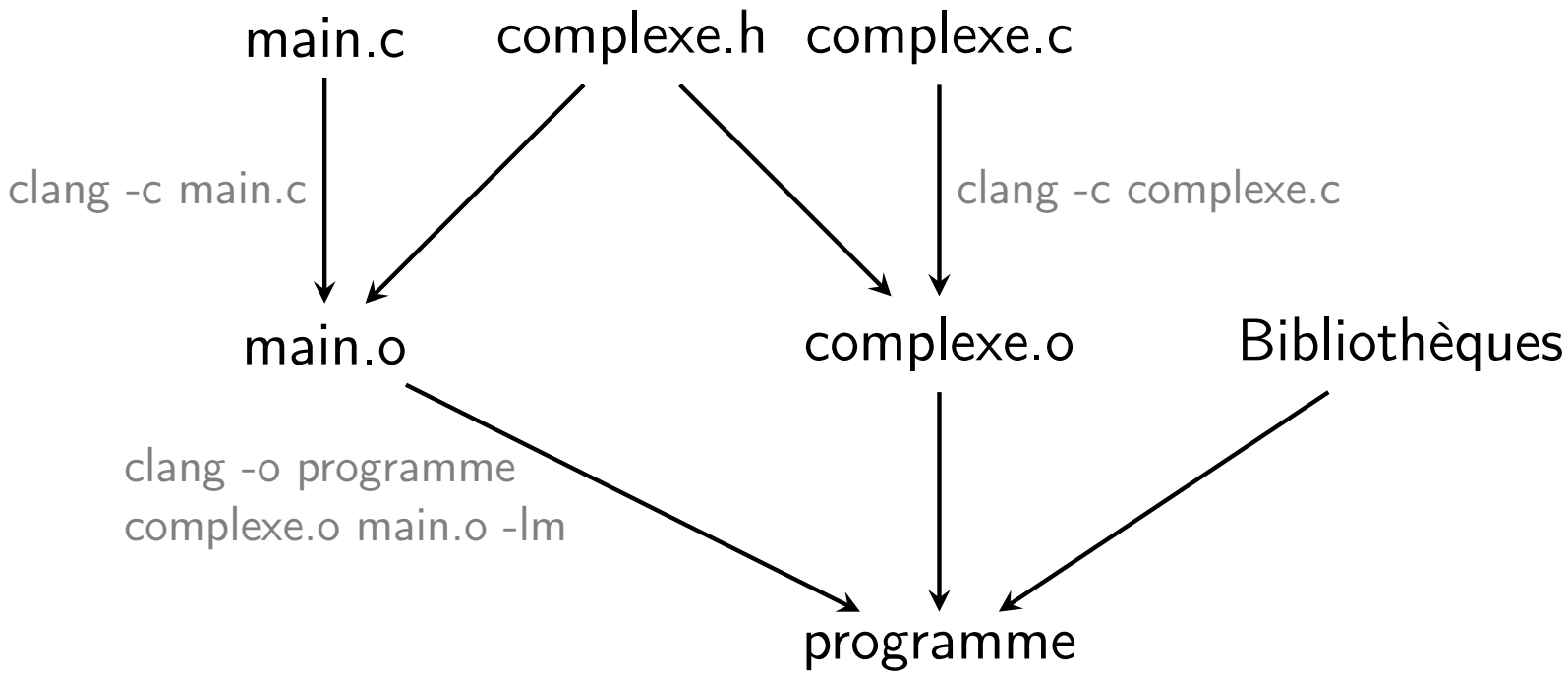
- édition de liens entre les fichiers objet `.o` et bibliothèques

```
clang -o programme complexe.o main.o -lm
```

Eviter de traduire inutilement

- un `.o` ne dépend que du `.c` associé et des `.h` qu'il inclus
- la traduction et l'optimisation sont lentes

Compilation d'un programme



Plusieurs commandes nécessaires à chaque mise à jour

Exemple

Première compilation

```
> clang -Wall -Werror -g -c complexe.c
> clang -Wall -Werror -g -c main.c
> clang -o programme complexe.o main.o -lm
```

On modifie `main.c` :

```
> clang -Wall -Werror -g -c main.c
> clang -o programme complexe.o main.o -lm
```

On modifie `complexe.c`

```
> clang -Wall -Werror -g -c complexe.c
> clang -o programme complexe.o main.o -lm
```

Effectivement, on ne recompile pas tout...

... mais gros besoin d'automatisation...

make

Outil d'automatisation de processus de construction

- on exprime les règles de fabrication pour chaque étape
- ne réexécute que les commandes nécessaires après modification

Règles contenues dans un fichier nommé `Makefile`

```
cible: liste des dépendances
    commande pour fabriquer la cible
```

Pas d'ordre particulier, mais la première règle est la cible par défaut

Exemple

Makefile

```
programme: complexe.o main.o
  gcc -o programme complexe.o main.o -lm

complexe.o: complexe.c complexe.h
  gcc -Wall -Werror -g -c complexe.c

main.o: main.c complexe.h
  gcc -Wall -Werror -g -c main.c
```

```
> make
clang -Wall -Werror -g -c complexe.c
clang -Wall -Werror -g -c main.c
clang -o programme complexe.o main.o -lm
> make
make: `programme' is up to date.
> touch main.c; make
clang -Wall -Werror -g -c main.c
clang -o programme complexe.o main.o -lm
```

Plan

Présentation de l'UE

Le langage C

Elements consitutifs du langage C

Programmation modulaire

Compilation séparée

Environnement de programmation

Documentation

Documentation des fonctions C dans la section 3 du manuel

```
man 3 nom_de_fonction
```

Dans la section *SYNOPSIS*

- noms des fichiers système à inclure
- prototype de la fonction cherchée
- bibliothèques avec lesquelles il faut faire l'édition de liens

Documentation sur le langage C lui même dans

- "Le langage C" de B.W. Kernighan et D.M. Ritchie, édité chez Dunod, ISBN-10 : 2100715771, ISBN-13 : 978-2100715770
- documentations comme celle référencée par la page de l'UE

Quel environnement ?

Textuel

- **vim** : extensible, ergonomique, 100% clavier, difficile
- **emacs** : extensible, ergonomie discutable
- atom, sublime text : bons intermédiaires, plus simples
- **gdb** ou **lldb** : complets, ligne de commande

IDE

- Eclipse, CLion
  - beaucoup (trop) de fonctionnalités
  - processus de construction caché
  - perte de la ligne de commande, lourd
- **VSCode**
  - simple, extensible et puissant
  - un peu de configuration textuelle à gérer

Outils de déboguage essentiels

`gdb` ou `lldb`, exécution dans un environnement contrôlé

- suivi de l'exécution
  - informations détaillées en cas d'erreur
  - points d'arrêt
  - exécution pas-à-pas
- visualisation
  - code source
  - état de la mémoire

D'autres seront vus

- `valgrind` ou *addresses sanitizer*
- `gprof`